

## Executive Summary

We propose building a scalable prototype web application using **Python 3.11** and **Django 6.1 LTS**, with **MySQL 8** as the database. The site will include typical pages (homepage, product catalog, cart, checkout, user account), support roles (guests, registered users, admins), and core features (authentication, CRUD for products/orders, search, file uploads, payments, and an admin interface). Non-functional requirements include horizontal scalability, caching, and strict security (HTTPS, OWASP best practices [2†L193-L202] ) with AA-level accessibility (WCAG 2.2) [39†L37-L40] . We recommend a modern tech stack: Django (6.1 LTS, supported until 2027 [18†L89-L92] ) on Python 3.11 (latest stable, ~10–60% faster than 3.10 [17†L51-L54] ), MySQL 8. Caching (Redis or Memcached) and a message queue (Celery with Redis/RabbitMQ) will improve performance. Containerization (Docker) with orchestration and CI/CD (e.g. GitHub Actions) will streamline development and deployment. We outline data models (ER diagram with User, Product, Order, etc.) mapped to Django models, REST API endpoints (e.g. `/api/products/` , `/api/orders/` with JWT or session auth [26†L555-L563] [26†L687-L695] ), and a cloud deployment architecture (load balancer, auto-scaling app servers, RDS with backups, S3 storage). We also describe the development workflow (Git branching, testing strategy with pytest), and best practices (secure settings, response caching, SEO-friendly URLs and sitemaps). Finally, we estimate effort (e.g. 2–3 developers, ~4–6 months for an MVP, ~8–12 months for full product) and costs (ballpark \$50K–\$100K for MVP, double for full). A prioritized roadmap with milestones and sample code (Django models, DRF serializers, Dockerfile/docker-compose) is included. References to official docs and tutorials are cited throughout for guidance.

## 1. Functional Requirements

- **Pages/UI:** The site will have a **Homepage** (promotional banners, featured items), **Product Catalog** (search/filter/list products), **Product Detail** pages (images, description, price), **Search Results** page, **User Account** (profile, order history), **Shopping Cart**, and **Checkout/Payment** pages. There will also be static pages (About, Contact, FAQ). An **Admin Dashboard** (using Django Admin) lets admins manage products, orders, and users [32†L55-L59] .
- **User Roles:**
  - *Guest/Visitor:* Can browse products and use search without logging in.
  - *Registered User/Customer:* Must sign up/login to purchase. Can view/update their profile, view order history, add/edit items in the cart, and checkout.
  - *Admin/Staff:* Can access the admin interface (Django Admin) to perform CRUD on products, categories, orders, and users [32†L55-L59] .
- **User Stories (Examples):** (format: “As a [role], I want [capability] so that [benefit]” [51†L1580-L1588] )
  - As a **guest**, I want to *search for products* by name or category, so that I can find items I’m interested in.
  - As a **customer**, I want to *register and login*, so that I can make purchases and view my order history.

- As a **customer**, I want to *add products to a cart* and *checkout with payment*, so that I can complete an online purchase.
- As a **customer**, I want to *upload a profile image or documents*, so that my account is complete.
- As an **admin**, I want to *create/update/delete products and categories*, so that I can manage the catalog.
- As an **admin**, I want to *view all orders* and change their status, so that I can process sales.
- As an **admin**, I want to *view site analytics and logs*, so that I can monitor usage and issues.

- **Core Features:**

- **Authentication & Authorization:** User signup, login, logout, password reset (via email). Email confirmation and two-factor optional (Django Allauth or custom). Role-based access control (use Django's `is_staff` / `is_superuser` and groups).
- **CRUD Operations:** Ability for admins to create/read/update/delete (CRUD) products, categories, orders, and users. Customers can manage their own profiles (update name, address, etc.) and view their orders.
- **Product Search/Filtering:** Full-text or keyword search (e.g. on product name/description), plus filtering by category, price range, etc. Use Django ORM or integrate with search backend if needed.
- **File Uploads:** Support uploading product images (multiple per product). Use image thumbnailing (Pillow). Store files on AWS S3 or a similar object storage via `django-storages` [54†L31-L39] .
- **Payments:** Integrate a payment gateway (e.g. Stripe or PayPal API) at checkout. Only the final amount is passed to the gateway; sensitive data (cards) handled by gateway.
- **Admin Interface:** Leverage Django's built-in admin site for managing data [32†L55-L59] . Customize ModelAdmin as needed.
- **Notifications:** Email confirmations for registration, order receipts. Could use Celery for sending emails asynchronously.
- **Security:** HTTPS enforced site-wide. CSRF protection (Django has built-in). Input validation and sanitization.
- **Session Management:** Use secure cookies. If building an API for SPAs, use token (JWT) or session auth with CSRF.
- **Other:** (Optional) Shopping cart persistence (db or session), product reviews/ratings, REST API for all resources.

## 2. Non-Functional Requirements

- **Scalability:** The architecture should support horizontal scaling. Use stateless application servers behind a load balancer (so new servers can be added easily). Database is a bottleneck, so use MySQL **read replicas** for scaling reads, and consider sharding or caching for heavy workloads. Use a CDN for static assets. Redis/Memcached caching reduces DB load. Plan for at least 1000 RPS (requests per second) capacity with autoscaling.
- **Performance Targets:** Aim for fast response times (e.g. 95th percentile < 500ms under target load). Optimize database queries (use indexes via `Meta.indexes` [34†L86-L93] , avoid N+1 queries with `select_related` / `prefetch_related` ). Use Django's QuerySet `explain()` to tune slow queries [34†L154-L158] . Use in-memory caches (Redis) for frequent data. Serve static/media via CDN or WhiteNoise for minimal latency.

- **Security:** Follow OWASP top 10. Use HTTPS exclusively (HSTS, `SECURE_SSL_REDIRECT=True`). Set `SESSION_COOKIE_SECURE` and `CSRF_COOKIE_SECURE` to True [\[2†L215-L223\]](#). Validate hosts ( `ALLOWED_HOSTS` ). Protect against SQL injection (Django ORM does parameterization) and XSS (auto-escaping in templates). Use `X-Frame-Options` header (Django has middleware). Use strong password hashing (Django's PBKDF2 by default). Implement rate-limiting (DRF throttle or Nginx) to prevent brute force and DoS.
- **Accessibility:** Follow **WCAG 2.2 AA** guidelines [\[39†L37-L40\]](#). Ensure semantic HTML, proper headings, ARIA labels, alt text for images, keyboard navigability, and sufficient color contrast. Django's default admin and forms adhere to accessibility best practices [\[39†L29-L33\]](#). Test with tools like Axe or WAVE.
- **Localization:** Not required in this scope (assume English-only). Ensure code can be internationalized (Django's `i18n`) if needed later.
- **Reliability/Availability:** Design for high availability: use multiple app servers and DB replicas. Implement automated backups (daily RDS snapshots, WAL logs). Plan failover (multi-AZ RDS). Include health checks (load balancer and application).

### 3. Technology Stack

- **Programming Language & Framework:** Python 3.11 (stable, performance gains over 3.10 [\[17†L51-L54\]](#)). Django 6.1 LTS (latest feature release in 2026, support through Apr 2027 [\[18†L89-L92\]](#)). Django 6.x includes modern async support and security features.
- **Database:** MySQL 8 (InnoDB). Use Amazon RDS (or Google Cloud SQL/Azure MySQL) with Multi-AZ deployment. Configure InnoDB, proper connection pooling. Use `mysqlclient` or `PyMySQL` driver. Apply schema migrations via Django ORM.
- **Front-end:** Primarily Django templates (server-rendered HTML). Optionally, use a modern JS framework (React or Vue) if a rich SPA is needed, connecting via Django REST Framework (DRF). For a standard prototype, Django's templating is simplest. Use Bootstrap or TailwindCSS for UI (via CDN or static files).
- **Caching:** Redis (via `django-redis`) or Memcached for caching pages, querysets, and sessions. Redis is recommended for its rich feature set (and can also serve as a message broker) [\[50†L24-L31\]](#).
- **Message Queue:** Celery for background tasks (email sending, order processing). Use Redis or RabbitMQ as the broker/back-end. Redis is easier to start with (and can double as cache) [\[50†L24-L31\]](#).
- **File/Media Storage:** Use AWS S3 (or compatible service) for static and media files. The `django-storages` package (with `boto3`) provides an S3 storage backend [\[54†L31-L39\]](#). Alternatively, store on the server with WhiteNoise for static files (not scalable long-term).
- **CI/CD:** Use Git for version control. CI via GitHub Actions/GitLab CI or Jenkins: run tests on push, lint code, build Docker image. CD pipeline can deploy to staging/production (e.g. push Docker

image to AWS ECR, then update ECS/EKS or EC2 auto-scaling group). Infrastructure as code (Terraform/CloudFormation) for reproducible infra.

- **Containerization:** Docker for development and production. Write a Dockerfile (base Python image) and use Docker Compose for local dev (services: web, db, redis). For production, use a container platform (AWS ECS/EKS, Docker Swarm, or bare EC2 with Docker) behind a load balancer [22†L155-L163] .
- **Monitoring & Logging:** Use Prometheus + Grafana to collect metrics (CPU, memory, request stats) from app servers. Use `django-prometheus` integration or custom instrumentation. Use Sentry (or similar) for error tracking (uncaught exceptions). Centralize logs (stdout to a log aggregator or ELK stack). Configure alerts for errors/performance issues.
- **Authentication:** Django's built-in auth for session-based login. For APIs, use DRF's authentication (SessionAuth for browser clients [26†L555-L563] or JWT/OAuth2 for token auth). We can use **DRF Simple JWT** or **Django OAuth Toolkit** for token/OAuth2 support [26†L687-L695] .
- **Other:** Use `django-environ` or Python-decouple to manage settings via environment variables. Use `python-decouple` or native environment via Docker secrets. Use `gunicorn` or `uvicorn/gunicorn` as the WSGI/ASGI server. Use Nginx as a reverse proxy (SSL termination) in production.

## 4. Key Python/Django Packages

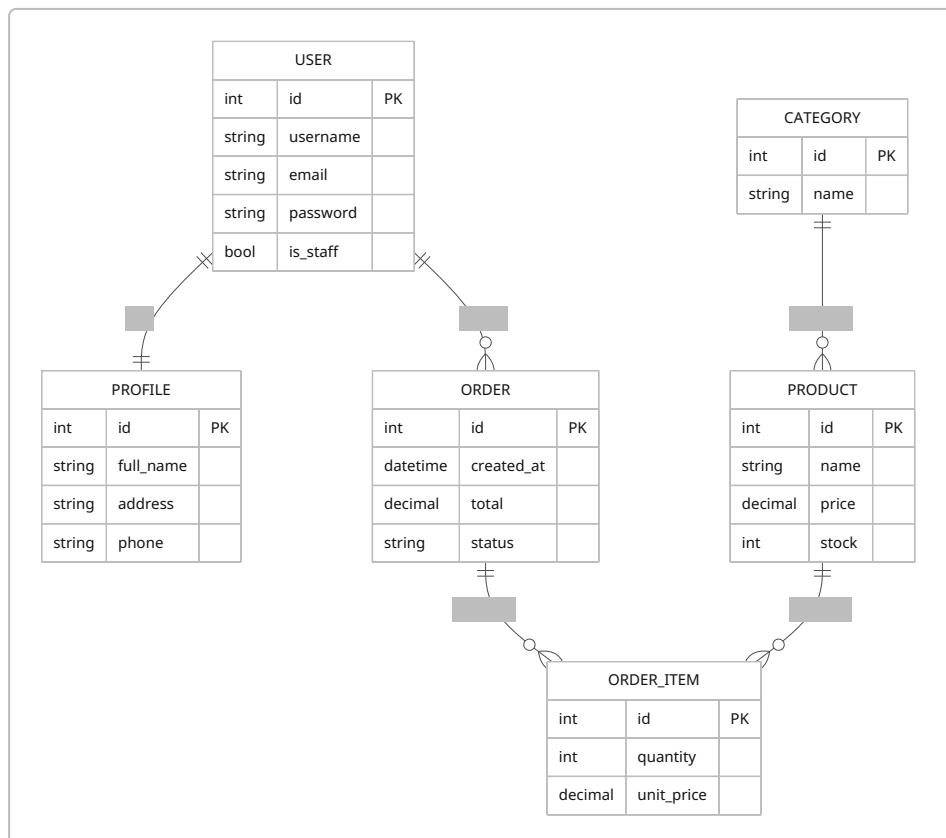
- `Django` – the web framework.
- `djangorestframework` (DRF) – for building RESTful APIs and serializers.
- `django-allauth` or `django-registration` – for user registration, login, social auth.
- `django-cors-headers` – enable CORS if a separate frontend (React/Vue) calls the API.
- `django-crispy-forms` (optional) – nicer form rendering in templates.
- `django-environ` / `python-decouple` – manage settings via environment variables.
- `mysqlclient` – Python MySQL driver (for MySQL).
- `django-redis` – Redis cache/session backend integration.
- `celery` – asynchronous task queue.
- `redis` – Redis server client.
- `stripe` or `paypalrestsdk` – if integrating Stripe/PayPal payments.
- `django-storages[boto3]` – S3 storage backend [54†L31-L39] for static/media.
- `gunicorn` or `uvicorn` – WSGI/ASGI server.
- `whitenoise` – serve static files in production without Nginx (for simple deployments).
- `pytest`, `pytest-django` – testing framework.
- `flake8` / `black` – code style and linting.
- `Sentry-SDK` – error tracking.
- (Optional) `django-oauth-toolkit` – for OAuth2 support if needed [26†L687-L695] .
- (Optional) `django-filter` – for easy queryset filtering in DRF or views.

Each package above serves a specific purpose (e.g. `django-redis` for caching, `django-storages` for S3, `celery` for background jobs, etc.).

## 5. Database Design (ER Diagram & Models)

We design a typical e-commerce schema. Key entities/tables include:

- **User** (extends Django's `auth_user` or a custom User): fields: `id` (PK), `username`, `email`, `password`, `is_staff`, etc. (index on email for login).
- **Profile** (1:1 with User): fields: `id`, `user_id` (FK), `full_name`, `address`, `phone`. Separates sensitive info.
- **Category**: `id` (PK), `name`, `slug`, `description`.
- **Product**: `id` (PK), `category_id` (FK), `name`, `slug`, `description`, `price` (decimal), `stock` (int), `image_url(s)`, `created_at`, `updated_at`. Index on `name` or `slug` for lookup.
- **Order**: `id`, `user_id` (FK), `total` (decimal), `status` (e.g. pending/shipped), `created_at`, `updated_at`.
- **OrderItem**: `id`, `order_id` (FK), `product_id` (FK), `quantity`, `unit_price`. (Denormalize price for history.)
- (Optional) **Cart**: separate from orders, to store current items before checkout (could be an Order with status "cart").
- (Optional) **Payment**: `order_id` (FK), `amount`, `method`, `transaction_id`, etc. (Often handled by external gateway, so may not store card data.)
- **Indexes & Relations:**
  - User 1—1 Profile.
  - Category 1—\* Product.
  - Product — (through OrderItem) Order.
  - User 1—\* Order.



In Django, these map to models with `ForeignKey` and `OneToOneField`. For example:

```
class Profile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE,
related_name='profile')
    full_name = models.CharField(max_length=100)
    address = models.TextField()
    phone = models.CharField(max_length=20)

class Category(models.Model):
    name = models.CharField(max_length=50, unique=True)

class Product(models.Model):
    category = models.ForeignKey(Category, on_delete=models.PROTECT,
related_name='products')
    name = models.CharField(max_length=100, db_index=True)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    stock = models.PositiveIntegerField(default=0)
    # e.g. image = models.ImageField(upload_to='products/')
    created_at = models.DateTimeField(auto_now_add=True)

class Order(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='orders')
    total = models.DecimalField(max_digits=10, decimal_places=2)
    status = models.CharField(max_length=20, default='pending')
    created_at = models.DateTimeField(auto_now_add=True)

class OrderItem(models.Model):
    order = models.ForeignKey(Order, on_delete=models.CASCADE,
related_name='items')
    product = models.ForeignKey(Product, on_delete=models.PROTECT)
    quantity = models.PositiveIntegerField()
    unit_price = models.DecimalField(max_digits=10, decimal_places=2)
```

We would also add `Meta.indexes` or `db_index=True` on fields frequently queried (e.g. `User.email`, `Product.name`, `Order.created_at`) [\[34†L86-L93\]](#). Use `select_related` / `prefetch_related` to optimize ORM queries (avoid N+1s) [\[34†L154-L158\]](#).

## 6. API Design

We will expose a RESTful JSON API (via DRF) for the main resources: Users, Products, Orders, etc. Endpoints could include:

- `GET /api/products/` – List all products (support filtering/query params)
- `GET /api/products/{id}/` – Product details
- `POST /api/products/` – Create product (Admin only)
- `PUT/PATCH /api/products/{id}/` – Update product (Admin)

- `DELETE /api/products/{id}/` - Delete product (Admin)
- `GET /api/categories/` - List categories
- `POST /api/categories/` - Add category (Admin)
- ...
- `GET /api/orders/` - List user's orders (or all orders if Admin)
- `POST /api/orders/` - Place a new order (authenticated user)
- `GET /api/orders/{id}/` - Order detail
- `PATCH /api/orders/{id}/` - Update order (admin could change status)
- `DELETE /api/orders/{id}/` - Cancel order
- `POST /api/auth/login/` - Obtain session or token.
- `POST /api/auth/logout/` - Logout.
- `POST /api/auth/register/` - Create new user account.

#### Request/Response Schemas (examples):

Example `GET /api/products/` response (JSON):

```
{
  "results": [
    {"id":1, "name":"T-Shirt", "price":19.99, "stock":50},
    {"id":2, "name":"Mug", "price":9.99, "stock":120},
    ...
  ],
  "count": 123
}
```

Example `POST /api/orders/` body:

```
{
  "user_id": 10,
  "items": [
    {"product_id": 1, "quantity": 2},
    {"product_id": 3, "quantity": 1}
  ],
  "total": 59.97
}
```

The API will use **authentication**. For a server-rendered app, Django's session-based auth (with CSRF tokens) can be used (Django REST Framework's `SessionAuthentication` [【26†L555-L563】](#)). For SPAs or third-party clients, we would use token-based auth (JWT or OAuth2). The [DRF docs](#) note support for SessionAuth and third-party packages like **django-oauth-toolkit** (recommended for OAuth2)

【26†L687-L695】. We might use [Simple JWT](#) for JSON Web Tokens. Permissions will enforce that endpoints like creating products/orders can only be done by authenticated users (and `is_staff` for admin actions).

**Rate Limiting:** Implement per-IP or per-user rate throttling to prevent abuse. DRF provides throttle classes ( `AnonRateThrottle`, `UserRateThrottle` ). A common setup might limit to e.g. 100 requests/minute per user. Alternatively, use NGINX or Cloudflare rate limiting.

## 7. Deployment Architecture

A typical cloud deployment might look like this:

```
flowchart LR
    subgraph "Public Network"
        LB[Load Balancer (AWS ALB)]
    end
    subgraph "Application Tier"
        App1["Django App Instance"]
        App2["Django App Instance"]
    end
    subgraph "Database Tier"
        RDS["MySQL RDS Multi-AZ"]
        Replica["Read Replica"]
    end
    subgraph "Cache/Broker"
        Redis["Redis/Memcached"]
        Rabbit["RabbitMQ"]
    end
    subgraph "Storage/External"
        S3["AWS S3 (static/media)"]
    end
    LB --> App1
    LB --> App2
    App1 --> RDS
    App2 --> RDS
    App1 --> Redis
    App2 --> Redis
    App1 --> Rabbit
    App2 --> Rabbit
    App1 --> S3
    App2 --> S3
```

- **Cloud:** Use AWS (or GCP/Azure) for managed services. For example, AWS Elastic Load Balancer (ALB) distributes traffic to a fleet of Dockerized app servers in an Auto Scaling Group.
- **Compute:** App servers running Django behind Gunicorn. These instances mount the Docker image or use ECS/EKS with auto-scaling based on CPU or request count. Health checks ensure only healthy instances serve traffic.



- **Database:** AWS RDS for MySQL with Multi-AZ for failover. Use Read Replicas for offloading read-heavy queries. Backup snapshots nightly and enable point-in-time recovery. Store schema migrations under version control (run via CI/CD).
- **Storage:** AWS S3 for static and media files, fronted by CDN (CloudFront) for low-latency global access. Serve static assets via CDN and media via signed URLs or public buckets as appropriate. Use `django-storages` to upload files to S3 [\[54†L31-L39\]](#) .
- **Cache/Queue:** AWS ElastiCache (Redis) for caching session/data and as Celery broker. AWS MQ or RabbitMQ on EC2 if needed.
- **Networking:** Enforce HTTPS via ALB (SSL cert via ACM). Backend servers in private subnets. Security groups restrict DB access to app servers only. Use environment variables or AWS Secrets Manager for credentials.
- **Backups & DR:** Automated DB snapshots (daily/weekly). Store code in GitHub (with automated backups). Enable CloudWatch monitoring and set alerts on error rates or high latency.
- **Monitoring:** Use Prometheus/Grafana on a separate stack (or AWS CloudWatch metrics), e.g. with `django-prometheus` . Integrate with Grafana dashboards. Use Sentry for exception alerts.
- **Delivery:** A CI/CD pipeline (GitHub Actions or Jenkins) builds Docker images on code commits, runs tests, and on success deploys to staging/production (push to ECR, update ECS task or trigger server refresh). Infrastructure can be managed with Terraform/CloudFormation (e.g. to provision ALB, ASG, RDS, etc.).

## 8. Development Workflow & Testing

- **Repository Structure:** Use Git. A typical layout:

```
myproject/
  manage.py
  requirements.txt
  Dockerfile
  docker-compose.yml
  /backend_app/      # Django project folder
    settings.py, urls.py, wsgi.py, etc.
  /apps/             # Django apps (e.g. accounts, shop)
  /frontend/         # (Optional) for React/Vue if separated
  /tests/            # Root-level tests (pytest style)
```

- **Branching:** Follow Git flow or similar: `main` (production-ready), `develop` (integration), `feature/*`, `hotfix/*`. Code reviews via pull requests to ensure quality.

- **Testing Strategy:**

- **Unit Tests:** Use `pytest` or Django's `unittest` for models, utility functions, views, serializers. For example, test model methods and edge cases.
- **Integration/API Tests:** Use Django's test client or `rest_framework.test.APIClient` to simulate requests to endpoints, checking JSON schemas and auth enforcement.
- **End-to-End Tests:** (Optional) Using Selenium or Cypress for critical flows (signup, checkout) if frontend is complex.
- *Sample Test Case (pytest):*

```
# tests/test_models.py
import pytest
from django.contrib.auth.models import User
from shop.models import Product

@pytest.mark.django_db
def test_product_creation():
    prod = Product.objects.create(name="TestItem", price=9.99,
stock=10)
    assert prod.id is not None
    assert prod.stock == 10

@pytest.mark.django_db
def test_order_total_calculation():
    user = User.objects.create_user('u1', 'u1@example.com', 'pass')
    order = Order.objects.create(user=user, total=0, status='new')
    prod = Product.objects.create(name="Item", price=5.00, stock=5)
    OrderItem.objects.create(order=order, product=prod, quantity=2,
unit_price=5.00)
    order.refresh_from_db()
    assert order.total == 0 # (if auto calc is implemented, else
update logic test)
```

- **Continuous Integration:** Configure CI (GitHub Actions) to run `pytest`, linting, and build Docker on each push. Fail builds on test failures. Use coverage reports to ensure key code paths are tested.

- **Code Review & QA:** Enforce code review. Have manual QA for UI/UX before major releases.

## 9. Best Practices: Security, Performance & SEO

- **Security Checklist:**

- Enforce HTTPS & HSTS (see Django security docs [2†L193-L202] ).
- Set `ALLOWED_HOSTS` and secure cookies.
- CSRF protection enabled (don't use `@csrf_exempt` unless necessary).
- Validate user input; avoid raw SQL (Django ORM is safe from SQL injection).
- Use strong password policies.
- Keep dependencies updated; apply security patches.
- Principle of least privilege for DB user.
- Sanitize file uploads; limit file size and types.
- Regularly audit logs and use Sentry to catch exceptions.

- **Performance Optimizations:**

- Add DB indexes for frequently queried fields (via `db_index=True`) [34†L86-L93] .
- Cache expensive views or fragments (using Django's cache framework with Redis) [35†L18-L24] .

- Use `select_related` / `prefetch_related` in Django queries to reduce DB hits [34†L219-L227] .
- Serve static assets (CSS/JS) with caching headers.
- Compress responses (GZip) via middleware or web server.
- Use a CDN for global static content.

- Profile endpoints (e.g. Django Debug Toolbar in dev, QuerySet.explain in production) and optimize hotspots.

#### • Accessibility & SEO:

- Use descriptive `<title>`, `<h1>` tags, and meta descriptions on pages.
- Ensure all images have `alt` text. Use semantic HTML (e.g. `<nav>`, `<main>`, `<footer>` ).
- Provide a `/sitemap.xml` (Django has `django.contrib.sitemaps` ) and `robots.txt` for search engines.
- Implement RESTful, SEO-friendly URLs (use slugs instead of numeric IDs in product URLs).
- Add Open Graph/Twitter meta tags for rich sharing.
- Ensure mobile-friendliness (responsive design).
- Implement structured data (JSON-LD) for products if needed (for rich search results).
- Page load speed: optimize images (compression), minimize JS/CSS. Use `django-compressor` or pipeline if needed.

## 10. Effort & Cost Estimates

*Assumptions:* Small team (2–3 developers, 1 QA, 1 PM), offshore rates (USD 25–50/hr). Timelines assume agile sprints (~2 weeks).

- **MVP:** Core features (auth, product catalog, basic cart/checkout, admin). Estimated **3–6 months**. ~1000–1500 developer hours. Cost roughly **\$30K–\$75K** (depending on region).
- **Full Product:** Adding extras (reporting, advanced search, notifications, extensive QA, performance tuning). ~6–12 months. Cost ~**\$60K–\$150K**.

These are rough ballparks. Costs vary by developer rates, infrastructure choices, and scope creep. Cloud hosting (AWS) might add ~\$100–\$500/month for moderate usage (small instances, RDS, S3 storage), more with scale and add-ons.

## 11. Roadmap & Milestones

1. **Sprint 1–2: Project Setup & Planning** (2–4 weeks) – Define detailed specs, set up repo, linting, CI/CD pipeline, Docker dev environment. Create DB schema and initial models.
2. **Sprint 3–4: User/Auth & Admin** (2–4 weeks) – Implement user registration/login (Django Allauth), user profiles, and admin site setup.
3. **Sprint 5–6: Product Catalog** (2–4 weeks) – Build Category/Product models and CRUD, product listing and detail pages.
4. **Sprint 7–8: Cart & Checkout** (2–4 weeks) – Implement shopping cart (session or model), order creation, and integrate payment gateway sandbox.
5. **Sprint 9–10: Order Management & Notifications** (2–4 weeks) – Finalize order processing, email confirmations (via Celery), order history for users.
6. **Sprint 11: Search, Filtering & File Uploads** (2 weeks) – Implement product search and image upload functionality.
7. **Sprint 12: API Development** (2 weeks) – Expose key endpoints via DRF, implement JWT or OAuth2.

8. **Sprint 13: Testing & Bugfixes** (2 weeks) – Write comprehensive tests, fix issues, performance profiling.
9. **Sprint 14: Deployment & Monitoring Setup** (2 weeks) – Deploy to cloud (stage), set up monitoring and logging (Prometheus, Sentry).
10. **Sprint 15+: Polishing & Launch** – UI tweaks, SEO (sitemaps), security audit, documentation, final review, launch.

Deliverables at each milestone include functioning features, documentation, and code reviews. Adjust timeline as needed.

## 12. Sample Code Snippets

**Django Models:** (see section 5 above for examples).

```
from django.db import models
from django.contrib.auth.models import User

class Product(models.Model):
    category = models.ForeignKey(Category, on_delete=models.PROTECT)
    name = models.CharField(max_length=100, db_index=True)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    stock = models.PositiveIntegerField(default=0)
    created_at = models.DateTimeField(auto_now_add=True)
```

**DRF Serializer & ViewSet:**

```
from rest_framework import serializers, viewsets, permissions
from .models import Product

class ProductSerializer(serializers.ModelSerializer):
    class Meta:
        model = Product
        fields = ['id', 'name', 'price', 'stock']

class ProductViewSet(viewsets.ModelViewSet):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
    permission_classes = [permissions.IsAuthenticatedOrReadOnly]
    # e.g. add filter_backends = [DjangoFilterBackend] for filtering
```

**Dockerfile:** (for the Django app)

```
# Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
```

```
# Collect static files if needed:
# RUN python manage.py collectstatic --noinput
CMD ["gunicorn", "myproject.wsgi:application", "--bind", "0.0.0.0:8000"]
```

**docker-compose.yml:** (development environment)

```
version: '3.8'
services:
  db:
    image: mysql:8
    restart: always
    environment:
      MYSQL_DATABASE: mydb
      MYSQL_USER: user
      MYSQL_PASSWORD: pass
      MYSQL_ROOT_PASSWORD: rootpass
    volumes:
      - db_data:/var/lib/mysql
  redis:
    image: redis:7
    restart: always
  web:
    build: .
    command: gunicorn myproject.wsgi:application --bind 0.0.0.0:8000
    volumes:
      - ./app
    ports:
      - "8000:8000"
    depends_on:
      - db
      - redis

volumes:
  db_data:
```

These snippets illustrate the core structure; in practice you would expand them (e.g. add static files handling, environment variables, etc.).

## References & Further Reading

- **Official Django docs:** (Security [\[2†L193-L202\]](#) , Admin [\[32†L55-L59\]](#) , ORM/Queryset [\[34†L86-L93\]](#) , Accessibility [\[39†L37-L40\]](#) )
- **Python 3.11 Release Notes:** highlights performance improvements [\[17†L51-L54\]](#) .
- **Django Download page:** supported versions and LTS (6.1 series) [\[18†L89-L92\]](#) .
- **DRF Documentation:** authentication schemes (Session, Token, OAuth2) [\[26†L555-L563\]](#) [\[26†L687-L695\]](#) .
- **Docker official blog:** benefits of containerizing a Django app [\[22†L155-L163\]](#) .
- **Caching with Django:** TestDriven tutorial on caching (Redis vs Memcached) [\[35†L18-L24\]](#) [\[50†L24-L31\]](#) .

- **django-storages docs:** Amazon S3 backend setup 【54†L31-L39】 .
- **User Stories:** Atlassian Agile guide (story format) 【51†L1580-L1588】 .

Recommended tutorials and guides: Django official tutorial, RealPython/Medium articles on Django REST, AWS deployment guides (e.g. AWS Builder or AWS EKS/ECS tutorials), and Celery documentation. These resources (cited and unofficial) can provide step-by-step instructions as needed.

---